

Follow-up Meeting

Anonymous Credentials

Aurelien Chassagne

April 1, 2026

Table of Content

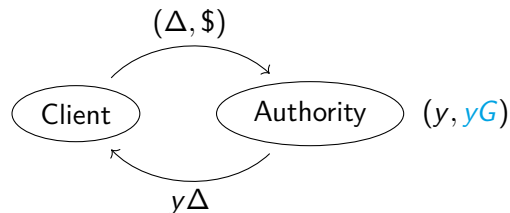
- Idea Overview
- Trivial Idea's Problem
- Preventing Leaking Node's Position
- Initial Credential
- Credential Update

Goal: Verifiable credential/signature at any hop **wihtout STTPs**

Overview

Goal: Verifiable credential/signature at any hop **without STTPs**

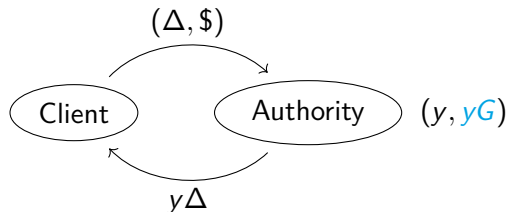
- 1 Client sends to authority a destination Δ (EC Point)
- 2 The authority signs the destination Δ



Overview

Goal: Verifiable credential/signature at any hop **without STTPs**

- 1 Client sends to authority a destination Δ (EC Point)
- 2 The authority signs the destination Δ



Header:



Credential

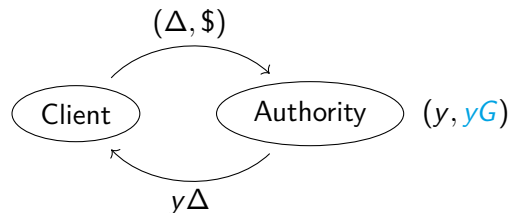
Overview

Goal: Verifiable credential/signature at any hop **without STTPs**

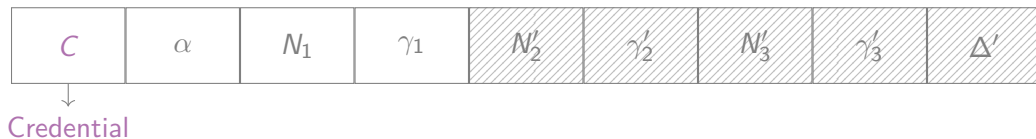
Some header chunks (with Δ)

$$e(H, Y) == e(C, G)$$

Authority's PK $\leftarrow$ Y C $\rightarrow$ Credential



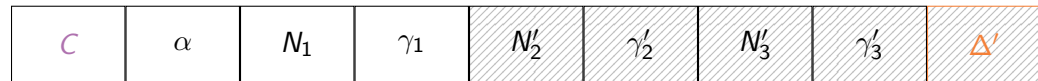
Header:



Trivial Idea

Idea: Using **only** the chunk containing Δ

Header:



↓
Credential

$$e(\Delta', Y) == e(C, G)$$

Trivial Idea

Idea: Using **only** the chunk containing Δ

Header:



Credential

$$e(\Delta', Y) == e(C, G)$$

Pros: the easiest and most effective

Cons: If able to distinguish the chunk containing $\Delta \Rightarrow$ Leaks node's position in the path

Prevent leaking node's position

Idea: Using **all chunks that could contain** 

Prevent leaking node's position

Idea: Using **all** chunks that could contain Δ

Header at node 1:

C_1	α_1	N_1	γ_1	N'_2	γ'_2	N'_3	γ'_3	Δ'
-------	------------	-------	------------	--------	-------------	--------	-------------	-----------

Header at node 2:

C_2	α_2	N_2	γ_2	N'_3	γ'_3	Δ'	X_1	X_2
-------	------------	-------	------------	--------	-------------	-----------	-------	-------

Header at node 3:

C_3	α_3	N_3	γ_3	Δ'	X'_1	X'_2	X_3	X_4
-------	------------	-------	------------	-----------	--------	--------	-------	-------

Prevent leaking node's position

Idea: Using **all** chunks that could contain Δ

There are **only 3 chunks** that could contain Δ (for a path of length 3)



$$e(\Delta' + N'_3 + N'_2, Y) == e(C, G)$$

Ccl: The credential C should be the **sum of these chunks** signed by the Authority secret key y

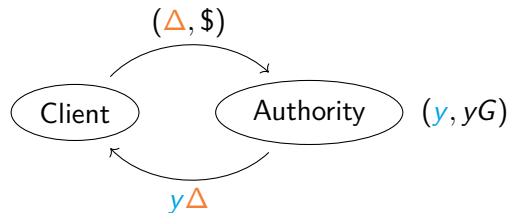
Initial credentials:

$$\begin{aligned} C_1 &= y (\Delta + N_3 + N_2 + (s_1 + s_2 + s_3)G_1 + (s_1 + s_2)G_3 + (s_1)G_5) \\ &= \underbrace{y\Delta}_{\text{from authority}} + yN_3 + yN_2 + (s_1 + s_2 + s_3)yG_1 + (s_1 + s_2)yG_3 + (s_1)yG_5 \end{aligned}$$

Credential Construction

Initial credentials:

$$\begin{aligned} C_1 &= y (\Delta + N_3 + N_2 + (s_1 + s_2 + s_3)G_1 + (s_1 + s_2)G_3 + (s_1)G_5) \\ &= \underbrace{y\Delta}_{\text{from authority}} + yN_3 + yN_2 + (s_1 + s_2 + s_3)yG_1 + (s_1 + s_2)yG_3 + (s_1)yG_5 \end{aligned}$$



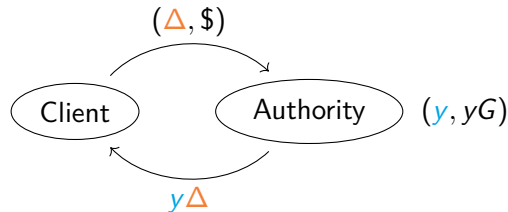
Credential Construction

Initial credentials:

$$\begin{aligned} C_1 &= y (\Delta + N_3 + N_2 + (s_1 + s_2 + s_3)G_1 + (s_1 + s_2)G_3 + (s_1)G_5) \\ &= \underbrace{y\Delta}_{\text{from authority}} + yN_3 + yN_2 + (s_1 + s_2 + s_3)yG_1 + (s_1 + s_2)yG_3 + (s_1)yG_5 \end{aligned}$$

Public parameters:

- yG
- $\{yN_i\} \forall \text{ mixnode } i$
- $\{yG_1, yG_3, yG_5\}$



Credential Update

Each mixnode update the credentials as:

$$C_{i+1} = C_i - s_i(yG_1 + yG_3 + yG_5 + yG_7) - yN_{i+1}$$

Credential Update

Each mixnode update the credentials as:

$$C_{i+1} = C_i - s_i(yG_1 + yG_3 + yG_5 + yG_7) - yN_{i+1}$$

Problem: Trivial updates (in/out differences) allow an adversary to link incoming and outgoing packets by comparing credential and chunk differences $\Rightarrow$ **Breaking unlinkability**.

Credential Update

Each mixnode update the credentials as:

$$C_{i+1} = C_i - s_i(yG_1 + yG_3 + yG_5 + yG_7) - yN_{i+1}$$

Problem: Trivial updates (in/out differences) allow an adversary to link incoming and outgoing packets by comparing credential and chunk differences $\Rightarrow$ **Breaking unlinkability**.

Solution: Rerandomized the credential at each update

$$e(C_{i+1}, G) = e(s_i^{-1}C_{i+1}, s_iG)$$

/!\ Rerandomization is not working /!\

Overhead

Computational overhead per packet:

(N = path length)

	EC Add	EC Mul	Pairing
Construction	$2N - 1$	N	0
Update	2	1	0
Verification	$N - 1$	0	2

→ 1x Once per packet

→ Nx Mandatory at each hop

→ (Nx) Optional at each hop

Communication overhead per packet = 1 EC point (i.e. 256 bits)

Memory overhead = $1 + N + M$ EC Points $\left\{ \begin{array}{ll} yG & \\ \{yN_i\}_{i \in [1, \dots, M]} & \forall \text{ mixnode} \\ \{yG_{1+2i}\}_{i \in [1, \dots, M]} & \forall \text{ hop} \end{array} \right.$

NB: And each client need to store all its credentials (one EC point for each destination)

- Check with code

/!\ **Need Pairing-based curve**

- Cannot use current Python lib (ECpy)
- Cannot use Elligator as reversible Hash-to-Curve (x-coord hash-to-curve)
- Charm-crypto, the state of the art Python lib for pairing, does not support hash-to-curve
⇒ Try with Rust libs ? Wait for Manuel's work ?

- Polishing the credential issuance

- Verify payment **OR** Blind signature is **easy**
- Verify payment **AND** Blind signature is **harder**
⇒ Investigate *Blind Verification*

Adding features:

- Blacklist destination
- Same/Different price by destination
- Validity period (i.e. infinite, time, usage, ...)